Wavelet Convolutions Are Memory-Bound: An I/O-Aware Formulation of WTConv

Anonymous authors
Paper under double-blind review

Abstract

Wavelet convolution (WTConv) has emerged as an increasingly popular drop-in replacement for standard convolutions, expanding a network's receptive field exponentially with the number of decomposition levels while keeping parameter count linear. However, because its reference implementation operates at an arithmetic intensity of only ≈ 1.63 FLOP/byte in `fp32`, the operator is severely memory-bound: in the 5×5 configuration in which it is actually deployed, it is 2.5–3.4× slower over a training step than the 7×7 depthwise convolution it is proposed to replace, despite performing comparable arithmetic. The cost is not computation but excessive data movement through high-bandwidth memory (HBM). We resolve this bottleneck with an exact, I/O-aware formulation based on three algebraic reformulations: (1) recomputing the multiplication-free Haar analysis in registers on chip, (2) collapsing the multi-level synthesis cascade into a single closed-form pass indexed by output coordinate bits, and (3) folding learned per-channel scales into the convolution weights. The resulting formulation is a reassociation of the operator that reduces HBM traffic by 2.5–2.6× across all levels. Measured over a full training step, a CUDA implementation is 3.71–4.35× faster than the reference in `fp32` and 2.68–3.09× in `fp16` while needing 1.83–2.31× less memory. This is enough to reverse the comparison that motivates the work: the fused layer trains 1.27–1.50× faster (`fp32`) and 1.40–1.76× faster (`fp16`) than the depthwise 7×7 convolution it replaces, at every decomposition level.

1 Introduction

Large receptive fields are important for modern convolutional networks, but obtaining them with conventional convolutions is expensive. Stacking small kernels expands the theoretical receptive field only linearly with depth, while directly increasing kernel size incurs parameter and arithmetic costs proportional to the kernel area (Luo et al., 2016; Ding et al., 2022; Liu et al., 2023). WTConv (Finder et al., 2024) offers an appealing alternative: it applies small depthwise convolutions across the progressively downsampled levels of a wavelet decomposition, so the receptive field grows exponentially with the number of levels while the parameter count grows only linearly. This makes WTConv a drop-in replacement for large depthwise convolutions and has produced accuracy and robustness gains in architectures including ConvNeXt and MobileNetV2 (Liu et al., 2022; Sandler et al., 2018).

Yet this advantage does not translate into execution speed. WTConv is deployed at k=5 — the kernel size WTConvNeXt uses — as a replacement for the 7×7 depthwise convolution of a ConvNeXt block. In that configuration the reference implementation is *slower than the convolution it replaces*: over a full training step it trails the depthwise 7×7 baseline by 2.46–3.42× in `fp32` and 1.53–2.20× in `fp16`, and at inference by 3.79–5.28× and 2.05–2.70× respectively. This is not explained by arithmetic: at k=5 the layer performs $58N$–$69N$ multiply–accumulates against $49N$ for the 7×7 depthwise baseline, a difference far smaller than the measured gap. An operator that increases wall-clock latency by this much offers a poor trade-off whatever its parameter count; closing the gap is the primary objective of this work.

We show that the discrepancy arises because FLOPs are the wrong cost model for this operator. The reference WTConv implementation has an arithmetic intensity of only 1.62–1.63 FLOP/byte in `fp32`, placing it deep

in the memory-bound regime on modern GPUs. Its execution is therefore dominated not by the cost of the Haar transform or the depthwise convolutions themselves, but by repeatedly materialising intermediate wavelet coefficients and reconstructions in high-bandwidth memory (HBM). Under a tensor-materialization I/O model, a forward evaluation incurs $17.75N$–$21.32N$ element reads and writes to global memory for an input containing N elements.

This observation suggests a different optimization target: rather than reducing arithmetic, we reformulate WTConv so that mathematical intermediates need not become memory-resident intermediates. We exploit three properties of the operator. First, Haar analysis is a multiplication-free butterfly and can be recomputed on chip inside the depthwise convolution. Second, linearity of Haar synthesis allows the entire L-level reconstruction recursion to be written as a single closed-form sum whose signs and coefficient addresses are determined by bits of the output coordinate. Third, the learned per-channel scales can be folded into the convolution weights. Together, these transformations preserve the same mathematical operator while eliminating the large intermediate tensors responsible for the majority of its data movement. The resulting formulation reduces modeled HBM traffic by 2.54–2.56× for $L = 1, \ldots, 5$.

We implement this formulation in CUDA and evaluate both inference and full training steps over a broad sweep of tensor shapes, decomposition levels, and precisions. For a full training step, the implementation is 3.71–4.35× faster than the reference in `fp32` and 2.68–3.09× faster in `fp16`, while using 1.83–2.31× less peak memory. More importantly, the systems reformulation reverses the practical comparison that motivates the work: over a training step the optimized WTConv is 1.27–1.50× faster in `fp32` and 1.40–1.76× faster in `fp16` than the depthwise 7×7 convolution it is proposed to replace, at every decomposition level tested. Training is the regime in which the comparison matters most, since it is where the materialised intermediates are both written and re-traversed; at inference the reversal is complete in `fp16` but not in `fp32`, where the fused layer remains within 3–17% of the depthwise baseline rather than ahead of it (Section 5.2).

Contributions.

- **An I/O cost model for WTConv.** We derive an element-level accounting of HBM traffic for the reference implementation, $Q_{\text{ref}} = 7N + \frac{43}{3}N(1 - 4^{-L})$, which is independent of the kernel size, and use a roofline analysis to show that WTConv lies roughly 31× below the compute-saturation ridge point in `fp32` at k=5. This identifies data movement, rather than arithmetic, as its dominant cost.
- **An algebraically exact, I/O-aware reformulation.** We combine register-resident Haar analysis, a closed-form bit-indexed synthesis over all decomposition levels, and scale folding to eliminate unnecessary HBM-resident intermediates. The resulting formulation reduces predicted forward-pass traffic by 2.54–2.56× across $L = 1, \ldots, 5$ while preserving the WTConv operator up to floating-point evaluation order.
- **End-to-end empirical validation.** Across 504 measured configurations, our CUDA implementation substantially reduces training and inference latency and peak memory relative to the reference WTConv implementation, and over a training step outperforms the depthwise 7×7 convolution WTConv is proposed to replace. We additionally verify numerical agreement for the forward output, input gradient, and all parameter gradients.

2 Background and Related Work

2.1 The WTConv operator

Let $\mathsf{X} \in \mathbb{R}^{B \times C \times H \times W}$ denote the input, and let $N = B \cdot C \cdot H \cdot W$ be its element count, the unit in which we quote every I/O cost. Let $*_{\text{dw}}$ denote depthwise convolution (one $k \times k$ kernel per channel, zero-padded to preserve resolution) and let $\odot$ denote per-channel scaling.

Let $\mathcal{A}$ denote one level of Haar analysis, which splits each channel into four half-resolution subbands $(\text{LL}, \text{LH}, \text{HL}, \text{HH})$, and let $\mathcal{A}^{\top}$ denote its synthesis, which inverts it exactly (1). For a tensor carrying a subband axis, let $[\,\cdot\,]_{\text{LL}}$ denote the low-pass band, $[\,\cdot\,]_{\text{H}}$ the three high-pass bands, and $\|$ their concatenation. WTConv preserves the channel count. It learns a $k \times k$ kernel and a per-channel scale at each level,

$(\mathbf{W}^{(\ell)}, \mathbf{s}^{(\ell)})$ for $\ell = 1, \ldots, L$, together with a full-resolution base convolution $(\mathbf{W}^{(0)}, b, \mathbf{s}^{(0)})$, giving $\mathcal{O}(Lk^2C)$ parameters in total. 1 states the operator as the reference implements it.

Algorithm 1 WTConv, reference implementation (Finder et al., 2024)

Require: input X, levels L, parameters $\{\mathbf{W}^{(\ell)}, \mathbf{s}^{(\ell)}\}_{\ell=0}^{L}$, bias b

$\mathsf{X}^{(0)} \leftarrow \mathsf{X}$ ▷ decomposition carrier
for $\ell = 1, \ldots, L$ **do** ▷ downward: decompose and filter
  $\mathsf{Y}^{(\ell)} \leftarrow \mathcal{A}\,\mathsf{X}^{(\ell-1)}$ ▷ Haar analysis
  $\mathsf{Z}^{(\ell)} \leftarrow \mathbf{s}^{(\ell)} \odot \left(\mathsf{Y}^{(\ell)} *_{\mathrm{dw}} \mathbf{W}^{(\ell)}\right)$ ▷ filter, then scale
  $\mathsf{X}^{(\ell)} \leftarrow \left[\mathsf{Y}^{(\ell)}\right]_{\mathrm{LL}}$ ▷ carry the *raw*, unfiltered low-pass band
end for
$R^{(L+1)} \leftarrow 0$
for $\ell = L, \ldots, 1$ **do** ▷ upward: reconstruct and accumulate
  $R^{(\ell)} \leftarrow \mathcal{A}^{\top}\left(\left(\left[\mathsf{Z}^{(\ell)}\right]_{\mathrm{LL}} + R^{(\ell+1)}\right) \,\middle\|\, \left[\mathsf{Z}^{(\ell)}\right]_{\mathrm{H}}\right)$
end for
return $\mathbf{s}^{(0)} \odot \left(\mathsf{X} *_{\mathrm{dw}} \mathbf{W}^{(0)} + b\right) + R^{(1)}$ ▷ base path + reconstruction

2.2 Haar transform differentiation

For each non-overlapping 2×2 block, the normalised Haar transform maps four input samples to four subband coefficients using the following matrix.

Proposition 1 (Symmetry and self-inversion)**.** $\mathbf{H} = \frac{1}{2}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$ *satisfies* $\mathbf{H}^{\top} = \mathbf{H}$ *and* $\mathbf{H}^2 = \mathbf{I}$*. Therefore,* $\mathbf{H}^{-1} = \mathbf{H}^{\top} = \mathbf{H}$.

The full analysis operator $\mathcal{A}$ applies $\mathbf{H}$ independently to every block, and synthesis applies $\mathcal{A}^{\top}$. For $\mathbf{y} = \mathbf{H}\mathbf{x}$, the chain rule gives

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \mathbf{H}^{\top}\frac{\partial \mathcal{L}}{\partial \mathbf{y}} = \mathbf{H}\frac{\partial \mathcal{L}}{\partial \mathbf{y}}. \tag{1}$$

Thus, backward through analysis applies synthesis, and backward through synthesis applies analysis. The Haar step needs no separate derivative; convolution and scale gradients are computed as usual.

2.3 The roofline

The roofline model (Williams et al., 2009) bounds attainable throughput by $\min(\pi, \beta I)$ where π is peak compute, β peak bandwidth, and I the arithmetic intensity in FLOP/byte. The ridge point $I^{\star} = \pi/\beta$ separates memory-bound from compute-bound operation. For the RTX A6000 used throughout, $\beta = 768$ GB/s and $\pi \approx 38.7$ TFLOP/s in `fp32`, so $I^{\star} \approx 50$ FLOP/byte. Any operator with $I \ll I^{\star}$ is bandwidth-limited: its runtime is governed by the bytes it moves rather than by how the arithmetic is scheduled. The bound is an upper bound on attainable throughput and not an equality, so this identifies traffic as the quantity to minimise without implying that runtime is proportional to it. The practical content of this paper is that WTConv is deeply in that regime, so minimising traffic is not merely one optimisation among many: it is the whole problem.

2.4 Kernel fusion and memory-bound operators.

It is well established that data movement, rather than arithmetic, is often the dominant cost in deep learning workloads (Ivanov et al., 2021; Wahib & Maruyama, 2014). FlashAttention (Dao et al., 2022; Dao, 2024) provides the canonical example: by exactly reformulating the computation so that intermediate results remain on chip, it substantially reduces the practical cost of attention. Our work applies the same principle to a different operator, but exploits additional algebraic structure: the fused transform is multiplication-free, symmetric, and self-inverse. These properties make recomputation inexpensive and allow analysis and

synthesis to exchange roles during backpropagation. General-purpose compilers (Ragan-Kelley et al., 2013; Chen et al., 2018; Sabne, 2020; **?**) can automate fusion across elementwise and reduction chains, but the reformulation developed in Subsection 4.2 (which collapse an L-step recursion into a closed-form, bit-indexed sum) depend on specific properties of the Haar basis that are not available to a general-purpose compiler.

3 WTConv is memory-bound

3.1 I/O cost of the reference implementation

The reference implementation expresses each level as a chain of framework primitives, every one of which reads its input from HBM and writes its output back (Figure 1a). We count elements crossing the HBM boundary: each stage reads its input tensor once and writes its output tensor once, weight traffic is $\mathcal{O}(Ck^2)$ and therefore negligible against $\mathcal{O}(N)$. Writing $N_\ell = N/4^{\ell-1}$ for the element count entering level ℓ:

- *Analysis*, per level: the Haar transform is executed as a grouped stride-2 `conv2d` against a constant $\pm\frac{1}{2}$ filter ($2N_\ell$); the depthwise convolution over the $4C$-channel coefficient tensor ($2N_\ell$); the scale multiply ($2N_\ell$). Total $6N_\ell$.
- *Synthesis*, per level: the cross-level low-pass add ($\frac{3}{4}N_\ell$); the concatenation of the summed low-pass band with the three high bands ($2N_\ell$); the `conv_transpose2d` ($2N_\ell$). Total $\frac{19}{4}N_\ell$.
- *Base path*: convolution ($2N$), scale multiply ($2N$), final add ($3N$). Total $7N$.

Using $\sum_{\ell=1}^{L} 4^{-(\ell-1)} = \frac{4}{3}(1-4^{-L})$,

$$Q_{\text{ref}} \;=\; \underbrace{7N}_{\text{base path}} \;+\; \underbrace{\left(6+\tfrac{19}{4}\right)}_{\text{per-level cost}} \cdot \underbrace{\tfrac{4}{3}\,N\big(1-4^{-L}\big)}_{\sum_{\ell=1}^{L} N_\ell} \;=\; 7N + \tfrac{43}{3}\,N\big(1-4^{-L}\big), \tag{2}$$

rising from $17.75N$ at $L = 1$ to $21.33N$ as $L \to \infty$; that is, evaluating the layer moves between 18 and 21 times the input tensor's worth of data through HBM. Note that k does not appear: the kernel size sets how much arithmetic each resident element receives, not how many elements cross the boundary. For a representative B=8, C=64, H=W=256 input, N = 33.6M elements (128 MiB in `fp32`) and the reference therefore moves roughly 2.9 GB per forward pass.

3.2 Arithmetic intensity of the reference implementation

The base convolution performs k^2 multiply–accumulates per element. At level ℓ, the depthwise convolution over the coefficient tensor costs a further k^2 per element of N_ℓ, and the analysis and synthesis each cost 4, since the reference executes both as 2×2 convolutions rather than as additions. Summing over levels with the same geometric factor as before,

$$\text{MAC} \;=\; N\big[k^2 + (k^2+8)\cdot\tfrac{4}{3}\big(1-4^{-L}\big)\big]. \tag{3}$$

For $k = 5$ and $L = 5$ this is $69.0N$ multiply–accumulates, or $137.9N$ FLOPs, against $4Q_{\text{ref}} = 85.3N$ bytes in `fp32`:

$$I_{\text{ref}} \;=\; \frac{137.9N}{85.3N} \;\approx\; 1.63 \text{ FLOP/byte}. \tag{4}$$

Against the ridge point $I^\star \approx 50$ (Subsection 2.3), WTConv therefore sits roughly 31× below the compute-saturation ridge point in `fp32`, using under 3.2% of the machine's arithmetic capability. Enlarging the kernel does not change this conclusion, only its margin: k multiplies the arithmetic while leaving Eq. 2 untouched, so even at $k = 5$ the operator remains an order of magnitude inside the memory-bound regime. Arithmetic is not the bottleneck: accelerating the transform itself would have little effect, whereas reducing memory traffic directly targets the dominant cost.

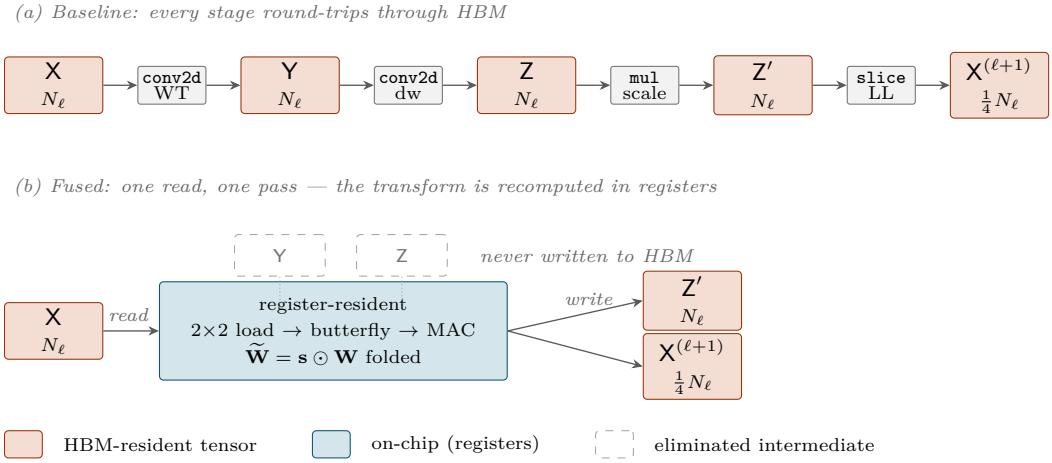

Figure 1: HBM dataflow for one WTConv decomposition level ℓ, with $N_\ell = N/4^{\ell-1}$ the number of elements entering that level. (a) The reference implementation expresses the level as a chain of framework primitives; every stage materialises a full-size tensor in HBM. (b) The fused formulation reads the input once, evaluates the Haar butterfly and the depthwise convolution on chip against weights that already carry the learned scale, and writes only the results. Y and Z are never written,

4 An I/O-Aware formulation

To eliminate the severe memory bottleneck of the reference WTConv, we derive an I/O-aware formulation based on three exact algebraic reformulations. These reformulations leave the underlying mathematical operator unchanged, differing from the reference implementation solely in floating-point evaluation order. Together, they eliminate all large intermediate subband tensors from high-bandwidth memory (HBM) by performing multi-stage computations directly in GPU registers and shared memory across all decomposition levels.

4.1 Fusing Haar analysis with depthwise convolution

For a 2×2 spatial patch $\left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right]$, the normalized 2D Haar analysis transform maps four input pixels to subband coefficients $(\mathrm{LL}, \mathrm{LH}, \mathrm{HL}, \mathrm{HH})$ via:

$$\begin{aligned} \mathrm{LL} &= \tfrac{1}{2}(a+b+c+d), & \mathrm{LH} &= \tfrac{1}{2}(a+b-c-d), \\ \mathrm{HL} &= \tfrac{1}{2}(a-b+c-d), & \mathrm{HH} &= \tfrac{1}{2}(a-b-c+d). \end{aligned} \tag{5}$$

Evaluating Eq. 5 requires only additions and scaling by $\frac{1}{2}$, with zero floating-point multiplications. In contrast, the reference implementation materializes the entire coefficient tensor $\mathsf{Y}^{(\ell)}$ in HBM via grouped convolutions with $\pm\frac{1}{2}$ filters. Each level runs Haar analysis, depthwise convolution, and learned scaling as separate passes. Each pass reads and writes an N_ℓ-element tensor, moving $6N_\ell$ elements in total. Our fused approach computes the Haar coefficients inside the depthwise convolution and folds the scale into its weights (Subsection 4.3). Because WTConv reaches only 1.63 FLOP/byte (Section 3), recomputing the coefficients is much cheaper than materialising them in HBM.

Naive fusion would recompute the transform for every convolution tap and load each input pixel k^2 times. Instead, each thread block transforms the 2×2 blocks for one output tile, including a coefficient-space halo of radius $R = (k-1)/2$, and stages the four subbands in shared memory. All taps reuse this copy, so each block is transformed once per tile. Except at the deepest level, the fused approach reads N_ℓ input elements, writes N_ℓ filtered coefficients for synthesis, and writes $\frac{1}{4}N_\ell$ unfiltered low-pass coefficients for the next level. It therefore moves $\frac{9}{4}N_\ell$ elements, versus $6N_\ell$ for the reference. The deepest level has no successor and moves $2N_L$.

4.2 Collapsing the synthesis cascade

The reference implementation executes reconstruction as an L-step sequential loop, where each level reads the lower-level reconstruction from HBM, adds it to its low-pass subband, and applies transposed convolution. This imposes L strict sequential dependencies and incurs an I/O cost of $\frac{19}{4}N_\ell$ per level.

Because Haar synthesis is linear, the low-pass carrier acts as a linear accumulator across levels. As a result, the multi-level synthesis cascade can be collapsed into a single closed-form pass across all L levels. Haar synthesis at level ℓ expands each subband coefficient over a $2^\ell \times 2^\ell$ pixel block using a fixed sign pattern determined by the spatial location of the output pixel within that block.

Proposition 2 (Bit-indexed single-pass synthesis)**.** *Let (y, x) denote an output pixel coordinate, and let level $\ell \in \{1, \dots, L\}$ have subband coefficients $(c_{\mathrm{LL}}^{(\ell)}, c_{\mathrm{LH}}^{(\ell)}, c_{\mathrm{HL}}^{(\ell)}, c_{\mathrm{HH}}^{(\ell)})$ at coarse grid location $(\lfloor y/2^\ell \rfloor, \lfloor x/2^\ell \rfloor)$. Define the coordinate parity signs at level ℓ as:*

$$s_y^{(\ell)} = (-1)^{\lfloor y/2^{\ell-1} \rfloor \bmod 2}, \qquad s_x^{(\ell)} = (-1)^{\lfloor x/2^{\ell-1} \rfloor \bmod 2}. \tag{6}$$

The reconstructed pixel value $R_{y,x}$ across all L levels is given by:

$$R_{y,x} = \sum_{\ell=1}^{L} 2^{-\ell} \Big(c_{\mathrm{LL}}^{(\ell)} + s_y^{(\ell)} c_{\mathrm{LH}}^{(\ell)} + s_x^{(\ell)} c_{\mathrm{HL}}^{(\ell)} + s_y^{(\ell)} s_x^{(\ell)} c_{\mathrm{HH}}^{(\ell)} \Big). \tag{7}$$

By evaluating Eq. 7 in a single pass, a GPU thread computing pixel (y, x) iterates from level L down to 1, addressing coarse coefficients via coordinate bit-shifts and obtaining signs through bitwise parity checks. This eliminates all intermediate low-pass tensors and sequential kernel dependencies. Furthermore, the base-path convolution output is folded directly into the final output store, eliminating a separate full-resolution tensor addition.

4.3 Folding the learned scale

The reference implementation applies learned per-channel scaling s_c after convolution ($\mathsf{Z}_c = s_c(\mathsf{X} *_{\mathrm{dw}} \mathbf{W})_c$) as a standalone elementwise pass. At an arithmetic intensity of $\frac{1}{8}$ FLOP/byte in `fp32` (one multiply per element read and written), this pass severely limits memory throughput.

By the linearity of convolution, the scale factor is folded into the depthwise weights and, on the base path, into its bias:

$$\widetilde{\mathbf{W}}_c = s_c \odot \mathbf{W}_c, \qquad \widetilde{b}_c = s_c\, b_c, \tag{8}$$

so that $\mathsf{Z} = \mathsf{X} *_{\mathrm{dw}} \widetilde{\mathbf{W}} + \widetilde{b}$ is computed in a single pass with zero extra runtime cost.

During backpropagation, exact parameter gradients with respect to the unscaled weight $\mathbf{W}_c$, bias b_c, and scale s_c are efficiently recovered via:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_c} = s_c \frac{\partial \mathcal{L}}{\partial \widetilde{\mathbf{W}}_c}, \qquad \frac{\partial \mathcal{L}}{\partial b_c} = s_c \frac{\partial \mathcal{L}}{\partial \widetilde{b}_c}, \qquad \frac{\partial \mathcal{L}}{\partial s_c} = \left\langle \frac{\partial \mathcal{L}}{\partial \widetilde{\mathbf{W}}_c}, \mathbf{W}_c \right\rangle + \frac{\partial \mathcal{L}}{\partial \widetilde{b}_c} b_c. \tag{9}$$

s_c enters both folds, so its gradient collects a term from each.

4.4 The resulting I/O cost

Collecting the reformulations across all levels $1 \dots L$, with $N_\ell = N/4^{\ell-1}$:

- *Base path*: the folded vendor convolution reads N and writes N ($2N$).
- *Analysis & Depthwise pass*, levels $1 \le \ell \le L$: reads N_ℓ, writes N_ℓ convolved coefficients and $\frac{1}{4}N_\ell$ raw low-pass coefficients ($\frac{9}{4}N_\ell$ each, less the $\frac{1}{4}N_L$ the deepest level does not write).
- *Single-pass Synthesis*: reads all subband coefficients $\sum_{\ell=1}^{L} N_\ell$ and base-path output N, writing the final output N ($\sum_{\ell=1}^{L} N_\ell + 2N$).

Using $\sum_{\ell=1}^{L} N_\ell = \frac{4}{3}N\big(1-4^{-L}\big)$, the total memory traffic is:

$$Q_{\text{fused}} \;=\; 4N \;+\; \tfrac{13}{3}\,N\big(1-4^{-L}\big) \;-\; \tfrac{1}{4}N\,4^{1-L}. \tag{10}$$

Comparing this to the reference memory traffic Q_{ref} (Eq. 2), the theoretical reduction ratio in HBM data movement is:

$$\frac{Q_{\text{ref}}}{Q_{\text{fused}}} = \frac{7+\frac{43}{3}(1-4^{-L})}{4+\frac{13}{3}(1-4^{-L})} \xrightarrow[L\to\infty]{} \frac{64}{25} = 2.56. \tag{11}$$

As $L \to \infty$, the fused formulation achieves up to a 2.56× reduction in memory traffic over the reference implementation. Table 1 summarizes the exact element counts and traffic reduction ratios for decomposition levels $L = 1 \ldots 5$. Like Eq. 2, neither count depends on k, so the traffic argument of this section is stated once and holds at every kernel size.

Table 1: Elements crossing the HBM boundary per forward evaluation, in units of $N = B \cdot C \cdot H \cdot W$, from Eq. 2, Eq. 10, and Eq. 11. Both counts are independent of the kernel size k. The last row is a ratio of *traffic*, not of runtime; see the discussion below.

	Decomposition levels L				
	1	2	3	4	5
Reference, Q_{ref}	$17.75N$	$20.44N$	$21.11N$	$21.28N$	$21.32N$
Fused, Q_{fused}	$7.00N$	$8.00N$	$8.25N$	$8.31N$	$8.33N$
Traffic removed	2.54×	2.55×	2.56×	2.56×	2.56×

5 Results

5.1 Setup

All measurements are single-layer microbenchmarks on an RTX A6000, except the end-to-end networks in Subsection 5.4. We sweep $C \in \{32, 64, 128\}$, $H = W \in \{128, 256, 512\}$ at B=8, and $L \in \{1, \ldots, 5\}$ in `fp32` and `fp16`. Timings average 50 iterations after 20 warmups.

Baselines. WTConv (reference and fused) always runs at k=5. We evaluate plain depthwise convolutions under two protocols: *matched* (k=5) to isolate wavelet overhead, and *drop-in* (k=7) representing the standard ConvNeXt replacement (Liu et al., 2022). Table 7 details parameter counts and receptive fields.

Reporting. Table values are the geometric mean over the (C, H) sweep of the per-configuration ratio between the WTConv reference and the evaluated method. The reference is always 1.00×. Latency is reported as speedup (reference/method; higher is better) and memory as footprint fraction (method/reference; lower is better). Appendix A verifies numerical agreement.

5.2 Layer latency

Training step. Table 2 reports full training-step (forward and backward) latency. The reference operator loses to the convolution it replaces: the depthwise 7×7 convolution of a ConvNeXt block completes a training step 2.46–3.42× faster in `fp32` and 1.53–2.20× faster in `fp16`; at matched kernel size the margin widens to 4.89–6.79× (`fp32`, depthwise 5×5).

Fusion reverses this. The fused layer is 3.71–4.35× faster than the reference in `fp32` and 2.68–3.09× in `fp16`. The ratio grows with L and then flattens, because deeper decompositions give the reference more intermediates to materialise while Eq. 10 is nearly flat in L; the change from L=4 to L=5 is below 1% in both precisions, as Eq. 11 predicts. At every level and in both precisions this suffices to overturn the comparison above: against the depthwise 7×7 convolution it replaces, the fused layer trains 1.27–1.50× faster in `fp32` and 1.40–1.76× faster in `fp16`. It does not win against a depthwise convolution at its *own*

Table 2: Latency of a full training step (forward and backward) relative to the reference WTConv (1.00×; higher is faster).

Method	Precision	Decomposition levels L				
		1	2	3	4	5
Depthwise conv, k=5	`fp32`	4.89×	6.21×	6.60×	6.74×	6.79×
	`fp16`	6.28×	8.12×	8.67×	8.90×	9.07×
Depthwise conv, k=7	`fp32`	2.46×	3.13×	3.33×	3.40×	3.42×
	`fp16`	1.53×	1.97×	2.11×	2.16×	2.20×
WTConv, reference	`fp32`	1.00×	1.00×	1.00×	1.00×	1.00×
	`fp16`	1.00×	1.00×	1.00×	1.00×	1.00×
WTConv, **fused**	`fp32`	3.71×	4.23×	4.34×	4.35×	4.33×
	`fp16`	2.68×	3.02×	3.08×	3.09×	3.08×

Table 3: Forward latency of every method relative to the reference WTConv (1.00×; higher is faster).

Method	Precision	Decomposition levels L				
		1	2	3	4	5
Depthwise conv, k=5	`fp32`	7.35×	9.28×	9.87×	10.10×	10.24×
	`fp16`	9.79×	11.95×	12.54×	12.75×	12.89×
Depthwise conv, k=7	`fp32`	3.79×	4.79×	5.09×	5.21×	5.28×
	`fp16`	2.05×	2.50×	2.62×	2.66×	2.70×
WTConv, reference	`fp32`	1.00×	1.00×	1.00×	1.00×	1.00×
	`fp16`	1.00×	1.00×	1.00×	1.00×	1.00×
WTConv, **fused**	`fp32`	3.67×	4.20×	4.32×	4.35×	4.37×
	`fp16`	3.38×	3.60×	3.60×	3.57×	3.53×

kernel size, which remains 1.32–1.57× faster in `fp32` and 2.34–2.94× in `fp16`; that kernel moves $2N$ elements against the fused layer's $7N$–$8.33N$ and carries no decomposition at all, so the direction is expected. The speedup over the reference is *smaller* in `fp16` than in `fp32` because half precision halves the bytes moved by both implementations, and the reference, which moves more of them, benefits more.

Inference. Table 3 reports forward-only latency. Against the reference the picture is unchanged: the fused layer is 3.67–4.37× faster in `fp32` and 3.38–3.60× in `fp16`, and the reference remains slower than both plain convolutions in the table, including the depthwise 7×7 it is meant to replace, by 3.79–5.28× and 2.05–2.70×. The comparison against the baselines, however, does not reverse as cleanly as over a training step. Against the depthwise 7×7 convolution the fused layer wins in `fp16` (1.31–1.65×) but not in `fp32`, where it sits at 0.83–0.97×. The `fp32` shortfall is structural rather than incidental: a forward pass allocates no autograd tape, so the part of the saving that comes from removing tape-retained subband tensors is unavailable, and what remains must cover a $7N$–$8.33N$ traffic budget against the $2N$ of a single well-tuned vendor kernel. Over a training step both arms pay for the backward traversal and the advantage reappears. We therefore claim the reversal for training, where the operator's cost binds, and report inference as it measures.

5.3 Peak memory

Training step. Table 4 reports peak allocated memory over a training step as a fraction of what the reference WTConv allocates. The fused layer needs 0.43–0.55× the memory of the reference (a 1.83–2.31× reduction), and the saving is essentially flat in L beyond L=2, because the dominant saved allocation is the level-1 coefficient tensor: it holds N elements, three times as many as all deeper levels combined ($\sum_{\ell\geq2} N_\ell \to N/3$). The mechanism is the one Section 4 describes: every per-level subband tensor the

Table 4: Peak allocated memory over a training step, as a fraction of what the reference WTConv allocates (lower is better)

Method	Precision	Decomposition levels L				
		1	2	3	4	5
Depthwise conv, k=5	`fp32`	0.63×	0.44×	0.44×	0.44×	0.44×
	`fp16`	0.73×	0.51×	0.51×	0.51×	0.51×
Depthwise conv, k=7	`fp32`	0.63×	0.44×	0.44×	0.44×	0.44×
	`fp16`	0.73×	0.51×	0.51×	0.51×	0.51×
WTConv, reference	`fp32`	1.00×	1.00×	1.00×	1.00×	1.00×
	`fp16`	1.00×	1.00×	1.00×	1.00×	1.00×
WTConv, **fused**	`fp32`	0.55×	0.43×	0.44×	0.45×	0.45×
	`fp16`	0.55×	0.44×	0.45×	0.45×	0.45×

Table 5: Peak allocated memory for inference, as a fraction of what the reference WTConv allocates (lower is better)

Method	Precision	Decomposition levels L				
		1	2	3	4	5
Depthwise conv, k=5	`fp32`	0.42×	0.29×	0.29×	0.30×	0.30×
	`fp16`	0.65×	0.45×	0.46×	0.47×	0.47×
Depthwise conv, k=7	`fp32`	0.56×	0.39×	0.40×	0.40×	0.40×
	`fp16`	0.66×	0.46×	0.47×	0.47×	0.47×
WTConv, reference	`fp32`	1.00×	1.00×	1.00×	1.00×	1.00×
	`fp16`	1.00×	1.00×	1.00×	1.00×	1.00×
WTConv, **fused**	`fp32`	0.65×	0.50×	0.52×	0.53×	0.53×
	`fp16`	0.65×	0.51×	0.53×	0.54×	0.54×

reference writes to HBM is also retained by autograd until the backward pass, whereas the fused layer recomputes the Haar coefficients on chip. Against the plain convolution, the fused layer's training-step footprint is 0.87–1.02× that of the depthwise 7×7 convolution in `fp32`, and lower still in `fp16` (0.76–0.89×). An L-level decomposition therefore trains within a few percent of the memory budget of a single plain convolution, and below it in half precision.

Inference. Table 5 is the forward-only counterpart, with no autograd tape. The saving over the reference is smaller than in training (0.50–0.65× against 0.43–0.55×), which is the expected direction, since a large part of what fusion removes is tape-retained subband tensors that inference never allocates in the first place; what remains is the traffic-side saving of Eq. 10. The plain convolutions are correspondingly harder to match, since without a tape their footprint is close to the compulsory input plus output: the fused layer sits at 1.15–1.34× the depthwise 7×7 convolution's footprint in `fp32` and 0.99–1.15× in `fp16`, the residual being the filtered subband coefficients it must materialise between the analysis and synthesis passes.

5.4 End-to-end networks

To evaluate the operator at architectural scale, we benchmark ConvNeXt-T and WTConvNeXt-T (Finder et al., 2024) which substitutes depthwise convolutions for WTConv (k=5, $L \in \{5, 4, 3, 2\}$) using `torch.compile` at batch 64 and 224×224 resolution in `fp32` Table 6. Because our reformulation is algebraically exact, accuracy is identical to the reference model.

Table 6: End-to-end network throughput and peak allocated GPU memory, for a forward pass and for a full training step (forward, backward, and SGD update). ConvNeXt-T is the unmodified architecture with its depthwise 7×7 blocks; WTConvNeXt-T is the same architecture with every depthwise convolution replaced by WTConv at k=5 with $(5, 4, 3, 2)$ decomposition levels across the four stages, in the reference and in the fused implementation. Parenthesised values are relative to ConvNeXt-T and the last column is fused relative to the reference; ↑ marks the rows where larger is better and ↓ those where smaller is. Batch 64 at 224×224 in `fp32`, all three networks compiled with `torch.compile`, 20 warmup and 50 timed batches on the device of Subsection 5.1, with the memory counter reset after warmup so that it reflects a steady-state iteration. Unlike **??**, these are absolute measurements at a single shape rather than geometric means over a sweep.

		ConvNeXt-T	WTConvNeXt-T		Fused /
		(depthwise)	reference	**fused**	reference
Inference	Throughput (img/s) ↑	1394.9	420.7 (0.30×)	988.5 (0.71×)	2.35×
	Peak memory (MiB) ↓	596.8	887.4 (1.49×)	750.8 (1.26×)	0.85×
Training step	Throughput (img/s) ↑	247.7	166.3 (0.67×)	**263.0** (**1.06**×)	1.58×
	Peak memory (MiB) ↓	6933.2	9685.6 (1.40×)	7618.4 (1.10×)	0.79×

The reference WTConv imposes a severe bottleneck, restricting the network to 30% of ConvNeXt-T's inference throughput and 67% of its training throughput. Our fused implementation effectively mitigates this, achieving 2.35× and 1.58× the reference's inference and training throughputs, respectively. Crucially, WTConvNeXt-T equipped with the fused layer trains 1.06× faster than the baseline ConvNeXt-T, although inference throughput reaches only 0.71× of the baseline.

The fused formulation similarly reduces peak memory, cutting the reference WTConvNeXt-T's training and inference footprints to 0.79× and 0.85×. This bounds the network's memory overhead over ConvNeXt-T to 1.10× (training) and 1.26× (inference).

6 Conclusion

WTConv's difficulty was never its arithmetic. Expressed as a chain of framework primitives it runs at under 3.2% of a modern GPU's arithmetic capability, spending its time moving tensors that need never have existed. Accounting for those bytes exactly, and removing them with three algebraic identities (a register-resident multiplication-free analysis, a synthesis cascade collapsed into a bit-indexed closed form, and a scale folded into the weights), yields a reformulation whose cost is within a small constant of the compulsory traffic, and which trains 1.27–1.50× faster in `fp32` than the depthwise 7×7 convolution the operator was introduced to replace. The broader point is that an operator's asymptotic parameter or FLOP advantage says little about whether it is usable; for the growing class of layers built from cheap, structured, multi-stage transforms, the I/O cost model is the one that predicts practice.

Broader Impact Statement

This work reduces the time and energy required to evaluate an existing operator without changing its function; the primary effect is lower computational cost for practitioners already using WTConv. We see no application-specific ethical concerns beyond those already attached to efficient computer vision generally.

References

Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Q. Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 578–594, 2018.

Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pp. 16344–16359, 2022.

Xiaohan Ding, Xiangyu Zhang, Jungong Han, and Guiguang Ding. Scaling up your kernels to 31x31: Revisiting large kernel design in CNNs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11963–11975, 2022.

Shahaf E. Finder, Roy Amoyal, Eran Treister, and Oren Freifeld. Wavelet convolutions for large receptive fields. In Aleš Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol (eds.), *Computer Vision – ECCV 2024*, volume 15112 of *Lecture Notes in Computer Science*, pp. 363–380, Cham, 2024. Springer. doi: 10.1007/978-3-031-72949-2_21.

Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefler. Data movement is all you need: A case study on optimizing transformers. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 3, pp. 711–732, 2021.

Shiwei Liu, Tianlong Chen, Xiaohan Chen, Xuxi Chen, Qiao Xiao, Boqian Wu, Tommi Kärkkäinen, Mykola Pechenizkiy, Decebal Constantin Mocanu, and Zhangyang Wang. More ConvNets in the 2020s: Scaling up kernels beyond 51x51 using sparsity. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A ConvNet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11976–11986, 2022.

Wenjie Luo, Yujia Li, Raquel Urtasun, and Richard Zemel. Understanding the effective receptive field in deep convolutional neural networks. In *Advances in Neural Information Processing Systems 29 (NIPS 2016)*, pp. 4898–4906, 2016.

Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 519–530, 2013. doi: 10.1145/2491956.2462176.

Amit Sabne. XLA: Compiling machine learning for peak performance. Google Research, 2020. URL https://research.google/pubs/xla-compiling-machine-learning-for-peak-performance/.

Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520, 2018.

Mohamed Wahib and Naoya Maruyama. Scalable kernel fusion for memory-bound GPU applications. In *SC '14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 191–202, 2014. doi: 10.1109/SC.2014.21.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

Table 7: Receptive field and parameter count (per channel) of WTConv at k=5, against the depthwise convolution it replaces and the depthwise convolution that would match its receptive field. The k=7 baseline spans 7 pixels at every level, so from L=1 onward WTConv covers strictly more; matching the receptive field with a plain kernel costs parameters quadratically where WTConv costs them linearly, which is the trade the operator exists to make.

Quantity	Decomposition levels L				
	1	2	3	4	5
WTConv receptive field (k=5)	10	20	40	80	160
WTConv parameters / channel	131	235	339	443	547
Depthwise k=7 receptive field	7	7	7	7	7
Depthwise k=7 parameters / channel	50	50	50	50	50
RF-matched depthwise k	10	20	40	80	160
RF-matched parameters / channel	101	401	1601	6401	25601

Table 8: Numerical agreement with the reference implementation. Entries are maximum absolute deviation over the whole tensor, at L decomposition levels, for the forward output and input gradient. The fused kernels compute the same multilinear map with a different association order, so the residual is floating-point reassociation error alone.

Backend	dtype	L	forward	∇_X
Fused (CUDA)	`fp32`	1	2.4e-07	2.4e-07
Fused (CUDA)	`fp32`	2	2.4e-07	2.4e-07
Fused (CUDA)	`fp32`	3	2.4e-07	2.4e-07
Fused (CUDA)	`fp32`	4	2.4e-07	2.4e-07
Fused (CUDA)	`fp32`	5	2.4e-07	2.4e-07
Fused (CUDA)	`fp16`	1	2.0e-03	9.8e-04
Fused (CUDA)	`fp16`	2	2.0e-03	2.0e-03
Fused (CUDA)	`fp16`	3	2.0e-03	2.0e-03
Fused (CUDA)	`fp16`	4	2.0e-03	2.0e-03
Fused (CUDA)	`fp16`	5	2.0e-03	2.0e-03
Fused (CUDA)	`bf16`	1	1.6e-02	1.6e-02
Fused (CUDA)	`bf16`	2	1.6e-02	1.6e-02
Fused (CUDA)	`bf16`	3	1.6e-02	7.8e-03
Fused (CUDA)	`bf16`	4	1.6e-02	1.6e-02
Fused (CUDA)	`bf16`	5	1.6e-02	1.6e-02

A Supplementary measurements

Latency and peak memory, for both a training step and inference and under both baseline protocols of Subsection 5.1, are reported in Section 5; this appendix carries the receptive-field and numerical-agreement tables.

Table 8 quantifies the numerical agreement referenced in Subsection 5.1. The deviations are those of a k^2-term accumulation in the working precision and do not grow with L: the forward output and ∇_X sit at one unit in the last place of the working format at every level, and the parameter gradients *shrink* with depth because the level-ℓ weight gradient is a reduction over $4^{-(\ell-1)}$ as many terms. The base-path bias gradient ∇_b, which checks the bias fold of Eq. 8, is a reduction over all BHW positions at every level, so its deviation is set by the reduction width rather than by L.